

```
int arr[5] = {1, 2, 3, 4, 5};
for (int& i: arr)
{
    cout<<i*i<<" ";
}
cout<<endl;
return 0;
}
```

The compilation and execution steps of the program *pgm3.cc* using a range-based *for* loop are given below. The program finds and outputs the squares of the numbers in the array *arr*.

```
>> g++ -std=c++11 pgm3.cc
>> ./a.out
1 4 9 16 25
>> g++ -std=c++98 pgm3.cc
pgm3.cc: In function 'int main()':
pgm3.cc:6:15: warning: range-based
'for' loops only available with
-std=c++11 or -std=gnu++11
    for (int& i: arr)
        ^~~
pgm3.cc:6:15: error: forming reference
to reference type 'int (&)[5]'
```

We can also observe that when the program is compiled with the g++ option, '-std=c++98' gives an error. The same error would have appeared even if we compiled with the g++ option '-std=c++03'. The error is due to the fact that this feature was not available in versions older than C++11. I am not sure about C++ programmers, but seasoned Python programmers will definitely appreciate this type of *for* loop. Now let us move on to C++14.

C++14 standard

C++14 (official document ISO/IEC 14882:2014) was introduced in 2014. Just like C++03, C++14 also made minor changes to the language — mostly bug fixes to the C++11 implementation. No new keywords were added. Some of the features introduced by C++14 include variable templates, generic lambda functions,

binary-literal 0b or 0B as integer suffix, etc. In C++14 also, the keyword *export* remained unused and reserved for future use. The g++ compiler claims full support for all the feature in C++ standards up to C++14. By using the g++ option '-std=c++14', we can make sure that programs strictly conform to the C++14 standard.

C++17 standard

C++17 (official document ISO/IEC 14882:2017) was introduced in 2017. Though supposed to be a major version, compared to C++98 and C++11, C++17 introduced fewer number of changes to C++. No new keywords were added. In addition to the keyword *export*, the keyword *register* also is unused and reserved for future use in C++17. Some of the new features brought in by C++17 include lambda functions returning constant expressions, initialisers for *if* and switch statements, structured bindings, introduction of the *u8* character literal, etc. There were some changes in the expression evaluation rules. For example, the statement '*i* = *i*++ + 1;' would cause an undefined behaviour in standards prior to C++17. The somewhat confusing and irritating trigraph sequences were finally deprecated by this standard. An example for a trigraph sequence is '??<', which denotes the opening curly bracket (*{*). The g++ compiler claims almost full support for the features required by the C++17 standard. It is recommended that you use g++ version 7 or higher to obtain this level of support. We can use the g++ option '-std=c++17' to make sure that a program strictly conforms to the C++17 standard.

Now, consider the program *pgm4.cc* given below:

```
#include<iostream>
using namespace std;

int main()
```

```
{
    cout<<__cplusplus<<endl;
    return 0;
}
```

The compilation and execution steps of the program *pgm4.cc* when compiled as a C++17 program and a C++14 program are given below:

```
>> g++ -std=c++17 pgm4.cc
>> ./a.out
201703
>> g++ -std=c++14 pgm4.cc
>> ./a.out
201402
```

The output is different because the value of the macro *__cplusplus* is changed with every version. If you are working with different standards of C++, it is advisable that you print the value of this macro at the beginning of every program so as to make sure that you are working with the correct version.

C++20 standard

The latest standard of C++, informally called C++20 (official document ISO/IEC 14882:2020), was introduced in 2020. Deviating from tradition, whereby the release of a major standard is followed by the release of a minor standard, C++20 has brought in some major changes to C++. It is widely believed that C++20, like C++98 and C++11 before, will change the way C++ is perceived by programmers. C++20 has 81 keywords. Eight more keywords were added to the existing 73 keywords of C++17. They are *char8_t*, *concept*, *constexpr*, *constinit*, *co_await*, *co_return*, *co_yield* and *requires*. As mentioned earlier, we are not counting the 11 alternative representations for operators and punctuators as keywords. So remember, suppose if someone tells you that there are 92 keywords in C++20, you should not contest the claim.

The previously unused and reserved keyword *export* is again